

Chapter7 Notes

Unit VII: Advance Topics on JavaScript

[7 Hrs.]

1. Scope and Closures

Scope — where a variable is accessible

```
let globalVar = "I'm global";    // accessible everywhere

function demo() {
  let functionVar = "I'm local to this function";
  if (true) {
    let blockVar = "I'm local to this block only";    // let/const are block-scoped
    console.log(blockVar);
  }
  // console.log(blockVar); // Error - not accessible here
}
```

Scope type	Keyword	Real use
Global	any, outside functions	Config values, app-wide constants
Function	var, let, const inside function	Local calculations
Block	let, const inside {}	Loop counters, if-block temp values

Closures — function “remembers” variables from where it was created, even after outer function finished

```
// Real project: shopping cart counter that keeps private state
function createCartCounter() {
  let count = 0;    // private variable - not accessible from outside directly
  return function() {
    count++;
    return count;
  };
}
```

```

}

const addToCart = createCartCounter();
console.log(addToCart()); // 1
console.log(addToCart()); // 2
console.log(addToCart()); // 3

```

Real project use: closures power private state in modules, debounce/throttle functions (search box that waits before firing API call), React's `useState` internally relies on closure concepts.

```

// Real project: debounce function for search input (avoids firing API on every
// keystroke)

function debounce(fn, delay) {
  let timer;
  return function(...args) {
    clearTimeout(timer);
    timer = setTimeout(() => fn(...args), delay);
  };
}

const searchAPI = debounce((query) => console.log("Searching for:", query), 500);
// searchAPI called on every keyup, but actual search only fires 500ms after user stops
// typing

```

2. Error Handling and Debugging

try...catch...finally

```

// Real project: safely parsing JSON from an API response
function parseUserData(jsonString) {
  try {
    const data = JSON.parse(jsonString);
    console.log("Parsed:", data);
    return data;
  } catch (error) {
    console.error("Invalid JSON:", error.message);
    return null;
  } finally {
    console.log("Parsing attempt finished."); // always runs
  }
}

parseUserData('{"name":"Uttam"}'); // works fine
parseUserData("invalid json");    // caught, doesn't crash the app

```

Throwing custom errors

```
function withdraw(balance, amount) {
  if (amount > balance) {
    throw new Error("Insufficient balance");
  }
  return balance - amount;
}

try {
  withdraw(1000, 5000);
} catch (err) {
  alert(err.message); // "Insufficient balance" - shown to user instead of app
                      // crashing
}
```

Debugging techniques used in real projects

- `console.log()` / `console.table()` / `console.error()` — inspect values at runtime.
- Browser **DevTools breakpoints** — pause execution, inspect variables line by line.
- `debugger;` statement — pauses execution when DevTools open.

```
function calculateTotal(items) {
  debugger; // execution pauses here when DevTools is open
  return items.reduce((sum, i) => sum + i.price, 0);
}
```

3. DOM Manipulation

DOM (Document Object Model) — browser's tree representation of HTML page; JS uses it to read/change page content live.

Selecting elements

```
document.getElementById("cart");
document.querySelector(".product-card"); // first match
document.querySelectorAll(".product-card"); // all matches (NodeList)
```

Changing content/style

```
const title = document.querySelector("h1");
title.textContent = "New Title";
```

```
title.style.color = "blue";
title.classList.add("highlight");
title.classList.remove("hidden");
title.classList.toggle("active");
```

Creating and inserting elements

```
// Real project: adding a new product card dynamically
const list = document.getElementById("productList");
const newItem = document.createElement("li");
newItem.textContent = "New Product - Rs. 999";
list.appendChild(newItem);
```

Event handling

```
// Real project: "Add to Cart" button click
document.getElementById("addToCartBtn").addEventListener("click", function() {
    alert("Item added to cart!");
});

// Real project: live form validation as user types
document.getElementById("email").addEventListener("input", function(e) {
    const isValid = e.target.value.includes("@");
    e.target.style.borderColor = isValid ? "green" : "red";
});

// Real project: event delegation - handle clicks on dynamically added product cards
document.getElementById("productList").addEventListener("click", function(e) {
    if (e.target.classList.contains("delete-btn")) {
        e.target.closest(".product-card").remove();
    }
});
```

Event delegation — attach one listener to parent instead of many to children; works even for elements added later. Used heavily in real projects for dynamic lists (to-do apps, cart items, comment sections).

4. Asynchronous JavaScript

JS is single-threaded — async patterns let it handle time-consuming tasks (API calls, timers) without freezing page.

Callbacks (older style)

```
function fetchUserData(callback) {
  setTimeout(() => {
    callback({ name: "Uttam", id: 1 });
  }, 1000);
}
fetchUserData((user) => console.log(user));
```

Problem in real projects: multiple nested callbacks → “**callback hell**” (hard to read/maintain).

Promises (fixes callback hell)

```
function fetchUserData() {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      const success = true;
      if (success) resolve({ name: "Uttam" });
      else reject("Failed to fetch user");
    }, 1000);
  });
}

fetchUserData()
  .then(user => console.log("User:", user))
  .catch(err => console.error(err));
```

async/await (modern, most used in real projects — cleaner syntax over Promises)

```
async function loadUser() {
  try {
    const user = await fetchUserData();
    console.log("User:", user);
  } catch (err) {
    console.error(err);
  }
}

loadUser();
```

Real project example — fetching product list from an API using async/await:

```
async function loadProducts() {
  try {
    const response = await fetch("https://api.example.com/products");
    const products = await response.json();
    renderProducts(products);
  }
}
```

```

    } catch (error) {
        console.error("Failed to load products:", error);
    }
}

function renderProducts(products) {
    const list = document.getElementById("productList");
    list.innerHTML = products.map(p => `<li>${p.name} - Rs. ${p.price}</li>`).join("");
}

```

5. JSON and AJAX

JSON (JavaScript Object Notation) — lightweight data format used to exchange data between client and server

```

const jsonString = '{"name":"Uttam","age":25,"isAdmin":false}';

const obj = JSON.parse(jsonString);    // JSON string -> JS object
console.log(obj.name);                  // "Uttam"

const backToJson = JSON.stringify(obj); // JS object -> JSON string (used when sending
                                         data to server)

```

AJAX (Asynchronous JavaScript and XML) — update page data without full reload

Using Fetch API (modern, preferred in real projects):

```

// GET request - real project: loading product details
fetch("https://api.example.com/products/1")
    .then(response => response.json())
    .then(data => console.log(data))
    .catch(error => console.error("Error:", error));

// POST request - real project: submitting a login form via JS instead of full page
// reload
fetch("https://api.example.com/login", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ email: "uttam@example.com", password: "secret123" })
})
    .then(response => response.json())
    .then(data => console.log("Login response:", data));

```

Using XMLHttpRequest (older, still asked in exams):

```
const xhr = new XMLHttpRequest();
xhr.open("GET", "https://api.example.com/products", true);
xhr.onreadystatechange = function() {
  if (xhr.readyState === 4 && xhr.status === 200) {
    const data = JSON.parse(xhr.responseText);
    console.log(data);
  }
};
xhr.send();
```

Real project use: search-as-you-type suggestions, infinite scroll loading more posts, submitting forms without page refresh, live notifications — all rely on AJAX/fetch + JSON.

6. ES6 and Modern JavaScript

Feature	Old way	ES6+ way	Real use
Variables	<code>var</code>	<code>let / const</code>	Block scoping, avoids bugs
Functions	<code>function() {}</code>	<code>() => {}</code> (arrow)	Shorter syntax, no own <code>this</code> (useful in callbacks)
Strings	<code>"Hello " + name</code>	<code>`Hello \${name}`</code> (template literals)	Cleaner string building
Object/array copy	manual loop	<code>... spread/rest</code>	Merging objects, copying arrays
Destructuring	<code>var name = user.name</code>	<code>const { name } = user</code>	Extract values cleanly
Modules	<code><script></code> globals	<code>import / export</code>	Organizing code into files

```
// Template literals - real project: dynamic message rendering
const user = { name: "Uttam", cartTotal: 1500 };
console.log(`Hi ${user.name}, your cart total is Rs. ${user.cartTotal}`);

// Destructuring - real project: extracting fields from API response
const { name, email } = { name: "Uttam", email: "uttam@example.com", age: 25 };
```

```

console.log(name, email);

// Array destructuring
const [first, second] = ["Shoes", "Watch"];

// Spread operator - real project: adding item to cart without mutating original array
// (React pattern)
const cart = ["Shoes", "Watch"];
const newCart = [...cart, "Bag"]; // ["Shoes", "Watch", "Bag"]

const baseConfig = { theme: "dark" };
const userConfig = { ...baseConfig, fontSize: 16 }; // merge objects

// Rest parameter - real project: function accepting unknown number of arguments
function sum(...numbers) {
  return numbers.reduce((a, b) => a + b, 0);
}
console.log(sum(10, 20, 30)); // 60

```

Modules — `import/export` (real project code organization)

```

// utils.js
export function formatPrice(amount) {
  return `Rs. ${amount.toFixed(2)}`;
}
export const TAX_RATE = 0.13;

// main.js
import { formatPrice, TAX_RATE } from "./utils.js";
console.log(formatPrice(500 * (1 + TAX_RATE)));

```

```
<script type="module" src="main.js"></script>
```

Real project use: every modern JS project (React, Vue, Node.js backend) splits code into modules by feature (`cart.js`, `auth.js`, `utils.js`) instead of one giant file.

7. JavaScript Libraries (Overview)

Library/Framework	Type	Purpose	Real use
jQuery	Library	Simplifies DOM manipulation, AJAX, animations (older projects)	<code>\$(".btn").click(...)</code> , image sliders

Library/Framework	Type	Purpose	Real use
React	Library	Component-based UI building, virtual DOM	Facebook, Instagram-style SPAs
Angular	Framework	Full MVC framework, TypeScript-based	Large enterprise apps
Vue.js	Framework	Lightweight, easy-to-learn component framework	Small-to-medium SPAs, progressive adoption

jQuery example (still common in legacy real projects):

```
<script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
<script>
$(document).ready(function() {
    $("#addToCartBtn").click(function() {
        $("#cartCount").text(parseInt($("#cartCount").text()) + 1);
    });
});
</script>
```

React example (conceptual overview, not in-depth):

```
function ProductCard({ name, price }) {
    return (
        <div className="card">
            <h3>{name}</h3>
            <p>Rs. {price}</p>
        </div>
    );
}
```

Why frameworks matter in real projects: vanilla JS DOM manipulation becomes hard to manage as apps grow (many interacting UI pieces) — frameworks like React manage UI state and re-rendering automatically, which is why most modern production web apps use one of these instead of pure vanilla JS.

Quick Recap (Exam Points)

- Scope: global, function, block (`let / const` are block-scoped, `var` is function-scoped).
 - Closure = function retains access to outer variables after outer function returns; used for private state, debounce.
 - Error handling: `try/catch/finally`, `throw new Error()`.
 - DOM manipulation: `getElementById / querySelector`, `addEventListener`, event delegation.
 - Async JS: callbacks → Promises (`.then/.catch`) → `async/await` (cleanest, most used today).
 - JSON: `JSON.parse()` (string→object), `JSON.stringify()` (object→string).
 - AJAX: `fetch()` (modern) vs `XMLHttpRequest` (legacy) — updates page without reload.
 - ES6 features: `let/const`, arrow functions, template literals, destructuring, spread/rest, modules (`import/export`).
 - Libraries: jQuery (DOM/AJAX helper), React/Angular/Vue (component-based frameworks for large apps).
-

Possible Exam Questions

Short Answer / Conceptual

1. What is a closure in JavaScript? Explain with an example.
2. Differentiate between function scope and block scope.
3. Explain `try...catch...finally` with an example.
4. What is DOM? Explain any three DOM methods used to select elements.
5. What is event delegation? Why is it useful in real projects?
6. Differentiate between callbacks, Promises, and `async/await`.
7. What is JSON? Differentiate `JSON.parse()` and `JSON.stringify()`.
8. What is AJAX? Explain how `fetch()` is used to make a GET request.
9. Explain any four ES6 features with examples.
10. What is the difference between `import/export` modules and traditional `<script>` tags?
11. Differentiate jQuery from modern frameworks like React.

Long Answer / Programming

1. Write a JavaScript program demonstrating closures using a counter function.
2. Write code to fetch data from an API using `async/await` and display it in an HTML list, including error handling with `try/catch`.
3. Create an event delegation example where clicking a “delete” button on any dynamically added list item removes that item.
4. Write a program that validates a login form using DOM manipulation and displays error messages without page reload.

5. Demonstrate use of `Promise` by simulating an API call with `setTimeout` , using `.then()` and `.catch()` .
6. Write JavaScript code using destructuring and spread operator to merge two objects and extract specific fields.
7. Create a simple search-as-you-type feature using `debounce()` and DOM event listeners.

True/False or Fill in the Blanks

1. `let` and `const` are function-scoped like `var` . (False — block-scoped)
2. A closure allows a function to access variables from its outer scope even after the outer function has returned. (True)
3. `fetch()` returns a Promise. (True)
4. `JSON.stringify()` converts a JSON string into a JavaScript object. (False — that's `JSON.parse()`)
5. _____ statement is used to pause code execution for debugging in DevTools. (`debugger`)
6. _____ keyword pauses an `async` function until a Promise resolves. (`await`)